Android Transaction Guide

The **Android Payment Transaction Guide** provides a step-by-step walkthrough for implementing payment transactions using the TransactionManager class. It focuses on the following key areas:

- **Create the TransactionManager Object**: Define and configure merchant and terminal information objects required for transaction processing.
- **Prepare & Execute the Transaction**: Initialize the TransactionManager, prepare transaction requests, and execute them using the appropriate methods.
- **Implement Real-Time Updates**: Implement the TransactionCallback to receive real-time updates on transaction progress, success, and failure.
- **Response Handling**: The onTransactionSucceeded and onTransactionFailed methods handle payment outcomes by confirming success with transaction details or reporting failure with status, terminal state, and error info.
- **Canceling an Ongoing Transaction**: Safely cancel an ongoing transaction.

ON THIS PAGE

Prerequisites

Create the TransactionManager Object

Prepare & Execute the Transaction

Implement Real-Time Updates

Response Handling

Canceling an Ongoing Transaction

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.

Note: *If you don't have a Deluxe Merchant Account or Access Token, simply visit Merchant Services Get Started and contact us today.*

Create the TransactionManager Object

Step 1: Create a TransactionManager Object.

```
val transactionManager = TransactionManager(
  requireContext(),
  merchantInformation,
  deviceInformation,
  posAppDetails!!
)
```

Note: *If the merchant account is configured to support surcharges then you must explicitly set the enableSurcharge flag to true. This ensures that the surcharge amount is calculated and applied correctly during the transaction.*

Step 2: Prepare Merchant & Terminal Information

Define the MerchantInformation, TerminalInformation, and PosAppDetails objects required to perform a transaction.

Merchant information identifies the party conducting the transaction, while terminal information provides details about the specific Moby5500 device and related configurations.

```
val merchantInfo = MerchantInformation(
companyId: "000005",// This should be 000005.
mid: "",// This should be mid of your merchant account.
tid: "",//This Should be tid of your mechant account.
bankId: "",//This can be left empty as of now.
merchantDBA: "",//This should be dba of your merchant account.
accountId: "",//This should be accessToken of the merchant account.
routing : "",
bearerToken: "" // This should be  bearer token received from the
MerchatAuthorizationService.
```

bearerToken can be retrieved using your MerchantAuthorizationService class and merchant accounts can be retrieved by using the MerchantAccountService class.

```
 //This is the INGENICO MOBY 5500 bluetooth connection terminalinformation
val terminalInfo = TerminalInformation(
manufacturer = TerminalManufacturer.INGENICO,          // Manufacturer of the
terminal (e.g., INGENICO)
model = TerminalModel.MOBY5500,                        // Terminal model (e.g.,
MOBY5500)
connectionType = TerminalConnectionType.BLUETOOTH,     // Connection type used
(e.g., BLUETOOTH)
connectionStatus = TerminalConnectionStatus.UNKNOWN,   // Current connection status
(e.g., CONNECTED, DISCONNECTED, UNKNOWN)
terminalAppName = "DlxCrtAnd",                         // Terminal application name
(fixed value)
terminalAppVersion = "0001",                           // Terminal application
version (fixed value)
terminalName = "name of the Moby5500 device to do the transaction",
firmwareInformation = "",                             // Optional: Ingenico
terminal firmware version
```

The **terminalName** field is mandatory and must exactly match the exact name of the Ingenico terminal.

Also provide Pos App Details as provided in following example

```
var posAppDetails = PosAppDetails(posAppName = "Client app name",
      posAppVersion = "client app version"
)
```

Prepare & Execute the Transaction

Step 1: Prepare Transaction Request

Create a TransactionRequest object tailored to the type of transaction.

Initialize an AmountDetails object with default transaction values—amount, tip, cashback, fees, and tax—all set to zero. You can update this AmountInformation as needed based on your requirements.

```
var amountDetails = AmountDetails(
amount = 10.00,                     // Total transaction amount
adjustedAmount = 0.00,              // Adjusted amount after any modifications
(e.g., discounts)
tax = 0.00,                         // Tax applied to the transaction
tip = 0.00,                         // Tip amount added by the customer
convenienceFee = 0.00,              // Any additional convenience fee applied
serviceCharge = 0.00,              // Service charge if applicable
cashBack = 0.00,                    // Cashback amount requested (e.g., in debit
transactions)
provisionalAmountInCent = 1000L     // Provisional total amount in cents (e.g.,
$10.00 = 1000 cents)
)
```

```
//This is an example of a SALE transaction request.
var transReq = TransactionRequest(
amountDetails = amountDetails,                         // Object containing all
amount-related fields (e.g., total, tip, tax)
transactionType = TransactionType.SALE.getDescription(), // Type of transaction
(e.g., SALE, REFUND)
transactionSuperType = TransactionSuperType.CREDIT.getDescription(), // Super type
(e.g., CREDIT, DEBIT)
transId = "",                                     // Unique transaction ID (could be
generated client-side or left empty for auto-assignment)
stin = "",                                          // Optional: STIN (Sales Tax
Identification Number)
partialApproval = "0",                                 // "1" if partial approval
is allowed, "0" otherwise
reverseTransactionType = "",                           // Used if this is a
reversal transaction; empty for normal transactions
merchantCartItems = merchantCartItems,                 // List of sale product
```

Step 2: Perform Transaction

Call the performTransaction method with the prepared request to start the transaction. The TransactionManagerCallback will handle updates on the transaction's progress and outcome.

```
transactionManager.performTransaction(
applicationContext,//This should be application context
```

The performTransaction method encapsulates all necessary transaction details and includes the callback handler.

```
var transactionCallback = object : TransactionCallback {

override fun onTransactionProgressUpdate(
transactionStatus: TransactionStatus,
terminalStatus: TerminalConnectionStatus,
description: String,
data: Any,
terminalManager: TerminalManager
) {
// Handle transaction status based upon transaction status and terminal status
}

override fun onTransactionSucceeded(
transactionStatus: TransactionStatus,
transactionResponse: TransactionResponse
) {
// Handle transaction success logic here
```

Implement Real-Time Updates

Implement onTransactionProgressUpdate to receive various progress updates during transaction processing.

This callback method is triggered whenever there is a progress update during transaction processing. Updates may relate to terminal connectivity or transaction execution. The method receives the current transaction status, terminal status, a progress description, and any additional data. It also provides the corresponding terminal manager object for any required terminal interactions.

```
override fun onTransactionProgressUpdate(
transactionStatus: TransactionStatus,
terminalStatus: TerminalConnectionStatus,
description: String,
data: Any,
terminalManager: TerminalManager
) {
// Here you will get Tranaction status and terminal status as well. Based on this
you can perform some action or change UI
}
```

During a transaction, the onTransactionProgressUpdate(...) method is invoked multiple times to provide real-time updates on the transaction's progress and the terminal's current status.

The following values are passed to this method:

transactionStatus: Current status of the transaction - This will be always inProgress.

terminalStatus: Current terminal connection status or transaction status (e.g., connected, pairing, paymentGatewayStartedProcessing etc.).

description: A string that provides additional context about the current status. In some cases, this may be a plain message (e.g., "Waiting for card"), while in others it may be paired with structured data in the data field.

data: A generic object (Any) used to pass additional structured information relevant to the current status. For example:

- When **terminalStatus** is **pairingLedIndicatorsAvailable**, data contains raw LED status information from the TerminalManager object which handles the terminal connection.
- When **terminalStatus** is **bluetoothDiscoveryCompleted**, data includes a list of discovered bluetooth terminals.
- When **terminalStatus** is **waitingForAdditionalChargeConfirmation**, data contains the additional charge information such as amount and additionalChargeType.
- For other statuses, **data** might be null.

Some of the important transaction progress updates include:

TerminalConnectionStatus.PAIRING_LED_INDICATORS_AVAILABLE: This will come only if the transaction is requested on Ingenico Moby 5500 terminal and **data** field contain the different LED

successfully connected and ready for the transaction.

TerminalConnectionStatus.CARD_NOT_PRESENTED: This status is shown when the terminal is waiting for the customer to tap, insert, or swipe their card to proceed.

TerminalConnectionStatus.PAYMENT_GATEWAY_PROCESSING_STARTED: indicates that the card data is collected and sent to payment gateway for processing.

TerminalConnectionStatus.PAYMENT_GATEWAY_PROCESSING_COMPLETED: Indicates that the transaction is completed.

TerminalConnectionStatus.WAITING_FOR_INLINE_TIP: Indicates that the terminal has completed all card-related actions and is now awaiting the input of a tip amount before the transaction can be processed by the payment processor. This process is known as inline tipping, which occurs prior to finalizing the transaction. At this stage, the transaction manager object will wait for the tip input by invoking the appropriate callback method.

```
transactionManager.updateTransactionAmount(
context: Context,              // Application context
amountDetails: AmountDetails,  // Class containing all kinds of amounts, including
tip
tenderAmount: Double           // Used in case of CASH transactions )
```

TerminalConnectionStatus.WAITING_FOR_ADDITIONAL_CHARGE_CONFIRMATION: This status indicates that an additional charge is found during the transaction processing and transaction manager is waiting for the merchant confirmation to proceed further. The data parameter will hold the object corresponding to the type of additional charge applied.

If the merchant confirms the additional charge then the following method call must be invoked to resume the transaction processing.

```
self.transactionManager?.confirmAdditionalCharge(isConfirmed)
```

isConfirmed must be true if the user confirms the surcharge otherwise this must be passed as false.

TerminalConnectionStatus.ADDITIONAL_CHARGE_CONFIRMATION_FAILED: This status indicates that the user declined the additional charge confirmation.

In this case, the transaction will be cancelled automatically by the transaction manager.

TerminalConnectionStatus.ADDITIONAL_CHARGE_CONFIRMATION_FAILED_DUE_TO_TIMEOUT: This status is triggered when the merchant does not respond to the additional confirmation within 120 seconds timeout.

In this case, the transaction will be cancelled automatically by the transaction manager.

Response Handling

The onTransactionSucceeded and onTransactionFailed methods handle payment outcomes by confirming success with transaction details or reporting failure with status, terminal state, and error info.

Implement Transaction Success

The onTransactionSucceeded(...) method is triggered when a transaction has been successfully processed by the payment processor. It provides both the transaction status and a detailed response object containing all relevant transaction information. This method is typically used to confirm the successful payment to the user, generate or print receipts, and perform any post-transaction operations, such as updating the UI.

The **transactionStatus** confirms the completion of the transaction by the payment processor, while the **transactionResponse** includes the actual response from the payment processor, containing essential

```
override fun onTransactionSucceeded(
transactionStatus: TransactionStatus,
transactionResponse: TransactionResponse) {
//Handle on transaction succeeded with transactionStatus and transactionResponse
}
```

Transaction Failure Handling

The **onTransactionFailed(...)** method is triggered when a transaction could not be completed due to an error or interruption. This method provides the transaction status, the terminal's last known state, and an error message that describes the cause of the failure.

The **status** indicates the general failure of the overall transaction.

The **terminalStatus** helps identify the terminal's last known state at the time of failure (e.g., disconnected, card not presented), which could also be a reason for transaction failures. For example: bluetoothDeviceNotFound, pairingCancelled, pairingFailed.

The **error** string provides a description of the error that caused the transaction to fail.

```
override fun onTransactionFailed(
status: TransactionStatus,
terminalStatus: TerminalConnectionStatus,
error: String) {
//Handle on transaction Failed with  status,terminalStatus,error
}
```

Canceling an Ongoing Transaction

Step 1**:** Cancel Transaction

Invoke the cancelTransaction() method to start the cancellation process and make sure to use the same Transaction Manager instance that has been used to initiate the original transaction that you trying to cancel.

```
transactionManager?.cancelTransaction(context)
```

This method will start the cancel process asynchronously and response will be send back to the Transaction callback as part of the TransactionManager object.

Step 2: Handle Cancel Transaction Callbacks

```
override fun onCancelTransactionSucceeded() {

}
override fun onCancelTransactionFailed(
         error: String,
         terminalStatus: TerminalConnectionStatus
     ) {

}
```

The following 2 callback methods will be notified with the cancel transaction responses.

onCancelTransactionSucceeded(): This method is invoked when a transaction cancellation request completes successfully. It indicates that the transaction was canceled without any errors.

onCancelTransactionFailed(error, terminalConnectionStatus): This method is called when a transaction cancellation attempt fails. It provides information about the failure through the error message and reports the current connection status of the terminal via TerminalConnectionStatus.

Possible error scenarios represented by different terminal statuses are given below:

- **TRANSACTION_IS_PROCESSING_WITH_GATEWAY**: Indicates that the transaction is already being processed with the payment gateway. The terminal cannot cancel the transaction since it has moved past the cancellable stage.

a configuration process. The cancellation request cannot be processed until the configuration is completed.

- **TRANSACTION_CANNOT_BE_CANCELLED_DUE_TO_ONGOING_CANCELLATION**: Indicates that the cancellation request is already in progress, duplicate not allowed.
- **TRANSACTION_IS_NOT_CAPABLE_TO_CANCEL**: Transaction cannot be cancelled due to some issue usually if any exception occurred.

Note: *Please refer to the Complete SDK Reference for additional details Android SDK Reference Guide*

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave, Minneapolis, MN 55402.

© 2026 Deluxe Corporation